# 🎯 LAB 2: Modules & Package Management

## Week 3 | Build Reusable Code Components!

**Instructor:** Mohamed Amine Marzouk
**Difficulty:** Beginner
**Time Estimate:** 4-5 hours
**Outcome:** AA1, AA2 (Modules & Dependencies)

---

## 📍 WHERE WE ARE

Lab 1 taught you **HOW** Node.js works.
Lab 2 teaches you **HOW TO ORGANIZE** your code.

---

## 🎬 WHY THIS MATTERS

Imagine you're building a house:

```
❌ BAD WAY: Write ALL code in one file (server.js)
- server.js has 5,000 lines!
- Everything mixed together
- Can't reuse anything
- Nightmare to maintain

☑ GOOD WAY: Break into modules
- validators.js (checks if data is valid)
- helpers.js (utility functions)
- config.js (settings)
- Each file does ONE thing
- Reusable everywhere
- Easy to maintain
```

**Real companies** like Netflix, Uber, and Airbnb use this approach. Their apps have thousands of files, each with one responsibility.

---

## TASK 1: CREATE YOUR FIRST MODULE

### Step 1: Create folder structure

```
cd Lab2
mkdir utils
mkdir config
```

Step 2: Create `utils/validators.js`

```
// utils/validators.js
// REUSABLE VALIDATION FUNCTIONS

function validateEvent(event) {
  const errors = [];

  // Check title
  if (!event.title || event.title.trim() === '') {
    errors.push('Event title is required');
  } else if (event.title.length > 100) {
    errors.push('Title cannot exceed 100 characters');
  }

  // Check date
  if (!event.date) {
    errors.push('Event date is required');
  } else if (new Date(event.date) <= new Date()) {
    errors.push('Event date must be in the future');
  }

  // Check capacity
  if (!event.capacity || event.capacity < 1) {
    errors.push('Capacity must be at least 1');
  }

  return {
    isValid: errors.length === 0,
    errors: errors
  };
}

function validateEmail(email) {
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  return emailRegex.test(email);
}

function validatePassword(password) {
  if (password.length < 6) {
    return { isValid: false, message: 'Password must be at least 6 characters' };
  }
  return { isValid: true };
}

// EXPORT these functions so other files can use them
module.exports = {
  validateEvent,
  validateEmail,
  validatePassword
};
```

### Step 3: Create `utils/helpers.js`

```
// utils/helpers.js
// UTILITY HELPER FUNCTIONS

function formatDate(date) {
  return new Date(date).toLocaleDateString('en-US', {
    year: 'numeric',
    month: 'long',
    day: 'numeric'
  });
}

function calculateEventDuration(startDate, endDate) {
  const start = new Date(startDate);
  const end = new Date(endDate);
  const hours = (end - start) / (1000 * 60 * 60);
  return hours;
}

function getEventStatus(eventDate) {
  const now = new Date();
  const event = new Date(eventDate);

  if (event < now) return 'past';
  if (event.toDateString() === now.toDateString()) return 'today';
  return 'upcoming';
}

module.exports = {
  formatDate,
  calculateEventDuration,
  getEventStatus
};
```

### Step 4: Create `config/constants.js`

```
// config/constants.js
// CONFIGURATION CONSTANTS

const MAX_EVENT_TITLE_LENGTH = 100;
const MAX_EVENT_DESCRIPTION_LENGTH = 1000;
const MIN_EVENT_CAPACITY = 1;
const MAX_EVENT_CAPACITY = 10000;
const PASSWORD_MIN_LENGTH = 6;
const EVENT_STATUSES = ['upcoming', 'ongoing', 'completed', 'cancelled'];
const USER_ROLES = ['user', 'organizer', 'admin'];

module.exports = {
  MAX_EVENT_TITLE_LENGTH,
```

```
  MAX_EVENT_DESCRIPTION_LENGTH,
  MIN_EVENT_CAPACITY,
  MAX_EVENT_CAPACITY,
  PASSWORD_MIN_LENGTH,
  EVENT_STATUSES,
  USER_ROLES
};
```

---

## TASK 2: USE YOUR MODULES

Create `Lab2_UsingModules.js`

```
// Lab2_UsingModules.js
// HOW TO USE YOUR NEW MODULES

// IMPORT the modules you created
const { validateEvent, validateEmail } = require('./utils/validators');
const { formatDate, getEventStatus } = require('./utils/helpers');
const { MAX_EVENT_TITLE_LENGTH } = require('./config/constants');

console.log('🎯 TESTING YOUR MODULES\n');
console.log('='.repeat(50) + '\n');

// TEST 1: Validate an event
console.log('TEST 1: Validating an event\n');

const goodEvent = {
  title: 'Tech Conference 2024',
  date: '2024-03-15',
  capacity: 500
};

const validationResult = validateEvent(goodEvent);
console.log('Event:', goodEvent);
console.log('Is valid?', validationResult.isValid);
console.log('Errors:', validationResult.errors);
console.log();

// TEST 2: Validate a BAD event
console.log('TEST 2: Validating a BAD event\n');

const badEvent = {
  title: '',  // Empty!
  date: '2020-01-01',  // Past date!
  capacity: 0  // Invalid capacity!
};

const badValidation = validateEvent(badEvent);
console.log('Event:', badEvent);
console.log('Is valid?', badValidation.isValid);
```

```
console.log('Errors:');
badValidation.errors.forEach(err => console.log(`   ❌ ${err}`));
console.log();

// TEST 3: Format dates
console.log('TEST 3: Formatting dates\n');

const eventDate = '2024-03-15';
console.log('Raw date:', eventDate);
console.log('Formatted:', formatDate(eventDate));
console.log();

// TEST 4: Get event status
console.log('TEST 4: Get event status\n');

const pastEvent = '2020-01-01';
const upcomingEvent = '2025-12-31';

console.log(`Event on ${pastEvent}: ${getEventStatus(pastEvent)}`);
console.log(`Event on ${upcomingEvent}: ${getEventStatus(upcomingEvent)}`);
console.log();

// TEST 5: Validate email
console.log('TEST 5: Validating emails\n');

const emails = [
  'user@example.com',
  'invalid-email',
  'test@domain.co.uk'
];

emails.forEach(email => {
  const isValid = validateEmail(email);
  console.log(`${email}: ${isValid ? '✅ Valid' : '❌ Invalid'}`);
});

console.log('\n✅ All module tests completed!');
```

Run it:

```
node Lab2_UsingModules.js
```

---

## TASK 3: UNDERSTAND NPM (NODE PACKAGE MANAGER)

### Step 1: Initialize npm

```
npm init -y
```

This creates `package.json` - your project's configuration file.

### Step 2: Look at your `package.json`

```
{
  "name": "nodejs-lab2",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

### Step 3: Add scripts

Edit `package.json`:

```
{
  "name": "event-management-system",
  "version": "1.0.0",
  "description": "Node.js learning labs",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "lab2": "node Lab2_UsingModules.js",
    "test": "node --test"
  },
  "dependencies": {},
  "devDependencies": {}
}
```

Now you can run:

```
npm run lab2
```

Instead of:

```
node Lab2_UsingModules.js
```

---

## TASK 4: INSTALL YOUR FIRST PACKAGE

### Step 1: Install dotenv (for environment variables)

```
npm install dotenv
```

### Step 2: Create `.env` file

```
# .env
APP_NAME=Event Manager
APP_PORT=3000
NODE_ENV=development
```

### Step 3: Create `config/environment.js`

```
// config/environment.js
require('dotenv').config();

module.exports = {
  app_name: process.env.APP_NAME || 'Event App',
  port: process.env.APP_PORT || 3000,
  env: process.env.NODE_ENV || 'development'
};
```

### Step 4: Use it

```
const env = require('./config/environment');

console.log(`App: ${env.app_name}`);
console.log(`Port: ${env.port}`);
console.log(`Environment: ${env.env}`);
```

---

## TASK 5: MODERN ES MODULES (OPTIONAL BONUS)

If you want to use modern syntax (`import` instead of `require`):

### Step 1: Update `package.json`

```
{
  "type": "module",
  "scripts": {
    "lab2-modern": "node Lab2_Modern.mjs"
```

```
  }
}
```

### Step 2: Create `Lab2_Modern.mjs` (`.mjs` = ES Module)

```
// Lab2_Modern.mjs
// Using modern ES module syntax

import { validateEvent, validateEmail } from './utils/validators.mjs';
import { formatDate } from './utils/helpers.mjs';

console.log('🎯 MODERN ES MODULES\n');

const event = {
  title: 'Conference',
  date: '2024-03-15',
  capacity: 100
};

const result = validateEvent(event);
console.log('Valid?', result.isValid);
```

### Step 3: Convert validators to ES module

Create `utils/validators.mjs`:

```
// utils/validators.mjs (ES Module version)

export function validateEvent(event) {
  // Same validation code
}

export function validateEmail(email) {
  // Same code
}
```

---

## ☑ LAB 2 DELIVERABLES

Submit these files:

1. ☐ `utils/validators.js` - Validation functions
2. ☐ `utils/helpers.js` - Helper functions
3. ☐ `config/constants.js` - Constants
4. ☐ `Lab2_UsingModules.js` - Testing all modules
5. ☐ `package.json` - Project configuration
6. ☐ `.env` - Environment variables

**Bonus:** ☐ `Lab2_Modern.mjs` - ES modules version

---

## 🎁 BONUS CHALLENGES

### Bonus 1: Create More Validators

```
// Add these to validators.js
function validateUsername(username) { }
function validatePhoneNumber(phone) { }
function validateUrl(url) { }
```

### Bonus 2: Create Logger Module

```
// utils/logger.js
function log(message) {
  console.log(`[${new Date().toISOString()}] ${message}`);
}

module.exports = { log };
```

### Bonus 3: Understand Semantic Versioning

In package.json:

```
{
  "dependencies": {
    "express": "^4.18.0"
  }
}
```

- `4` = Major (breaking changes)
- `18` = Minor (new features)
- `0` = Patch (bug fixes)

---

## 🆘 TROUBLESHOOTING

**"Cannot find module"**

- Check file path
- Make sure file extension is correct

**"module.exports is not defined"**

- Make sure you're using CommonJS (not ES modules)

**".env variables not loading"**

- Check `require('dotenv').config()` is at top
- Check spelling of variable names

---

## 📚 WHAT YOU'VE LEARNED

- [x] How to organize code into modules
- [x] module.exports and require()
- [x] NPM and package.json
- [x] Installing packages
- [x] Environment variables
- [x] ES Modules (modern syntax)

---

## 🎯 KEY TAKEAWAY

**Modules are the building blocks of professional code.**

Every large application uses modules to organize code. Learn this well!

---

**Next Lab: Express.js Basic Server**

---

**Instructor:** Mohamed Amine Marzouk
**Email:** med.amine.mzk@horizon-tech.tn
**Last Updated:** January 25, 2026